Maksim Solovev

DevOps Engineer, 36 y.o.

13+ years in IT, 6+ years in DevOps · Russia, Moscow

Telegram: @artfaal
Email: sys.dll@gmail.com
GitHub: artfaal
LinkedIn: maksim-solovev

ABOUT ME

DevOps Expert Engineer. Running DevOps for the Jet Detective anti-fraud platform (Jet Infosystems). Built CI/CD, monitoring, and a management system for 160+ VMs from scratch.
Experience leading a team of 7, shipping releases to production on large government projects, and adapting services for Kubernetes.
Currently building the AI side of the platform: agents for routine work with an eval gate before deploy, MCP servers, and documentation used by both people and agents. A separate track — local models for the closed network, where data never leaves the perimeter.

WORK EXPERIENCE

DevOps Expert Engineer

Oct 2023 – present

Jet Infosystems

Jet Detective — a fraud prevention and financial monitoring platform (anti-fraud for banks, retail, and industry). Responsible for all product infrastructure.

Infrastructure & automation

- Developed JD-Gateway (Python) — a single source of truth for 160+ VMs in vSphere: lifecycle management, TTL, REST API, dynamic inventory for Ansible and Prometheus, Web UI
- Created an automated product installer supporting CentOS, RedOS, and Astra Linux
- Automated VM provisioning via Packer (4 OSes) and Ansible (10+ roles)

CI/CD

- Designed and maintain complex Jenkins pipelines for build and deploy
- Implemented parallel builds and dynamic worker creation in vSphere
- Integrated auto-tests into PR pipelines and introduced OWASP Dependency Check
- Migrated repositories from Bitbucket to GitLab while preserving CI/CD integrations

Monitoring & logging

- Built the monitoring stack from scratch: Telegraf + Prometheus + Grafana + Thanos
- Set up alerting, created dashboards, implemented a flexible notification system
- Deployed OpenSearch + Loki + Promtail + Logstash for logs

Security

- Implemented HashiCorp Vault (GitOps) for secrets management
- Developed an SSH CA with Web UI and LDAP authentication
- Addressed security audit findings for a banking client

LLM & agents

- MCP servers on top of documentation and infrastructure — shared context for the team and LLM agents
- Agent harness for DevOps routine: engine picked after comparing five frameworks, eval gate before deploy (grounding, security, tool selection), tracing of model calls
- PoC of a diagnostic agent on local models — incident investigation without sending network data to the cloud
- Vault MCP for agents: secret delivery via deny policies and token isolation

Custom tooling

- Telegram bot for infrastructure management with LDAP and audit logging
- Comprehensive backup system (PostgreSQL, Jenkins, OpenSearch, Kafka) to MinIO S3
- GitLab webhook server for MR automation
- Apache Superset for embedded BI dashboards in the product

- Documentation platform based on MkDocs with drift-detection

Team Lead DevOps

Dec 2019 – Oct 2023

Jet Infosystems

Projects: EGRN, Atlant (Testing Automation Platform). Led a DevOps team of 7.

EGRN project

- Managed test environment infrastructure (6+ environments, 1000+ VMs)
- Shipped releases to production — debugging under fire, log analysis
- Production MongoDB upgrade — playbooks, production-like environment, data migration

Atlant / TAP project

- System architect: design, implementation, and support of a test automation platform
- Adapted services for Kubernetes, deployed k8s clusters
- CI/CD consulting for internal projects, pre-sales for banking clients

System Administrator / Web Developer

Mar 2013 – Oct 2019

SaunaMaster → EOS Premium Spa Technologies

Started as the sole IT specialist in the company, grew from sysadmin to web developer.

Sysadmin (2013–2015)

- Designed small-business IT infrastructure from scratch: LAN, servers, IP telephony, CCTV

Web development (2015–2019)

- Commercial website development: DigitalOcean, Nginx, Python, Flask, MongoDB
- Built promotional sites for marketing campaigns

SELECTED CASES

01. JD-Gateway — single source of truth for 160+ VMs

Challenge: 160+ virtual machines across environments. Information scattered across Confluence, engineers' heads, and vSphere consoles.

Solution: Developed JD-Gateway in Python — a web app with REST API, vSphere integration, dynamic inventory for Ansible and Prometheus, TTL system, and Web UI.

Result: Creating a host used to take 30 minutes by hand through vSphere — now it's a couple of minutes. 14 people handle these operations themselves; the requests have stopped coming. TTL prevents environments from turning into zombies.

Lesson: *"Single source of truth" is not an architectural pattern — it's a cultural decision. The technical part is 30%; the rest is persuasion and convenience.*

02. Monitoring from scratch for an anti-fraud platform

Challenge: A product with zero monitoring. Issues were discovered when someone complained or an environment went down.

Solution: Built the full stack: Prometheus, Grafana, Thanos for long-term storage. Alerting with a flexible notification system. OpenSearch + Logstash for logs.

Result: Problems are visible before anyone complains. Dashboards became the primary diagnostic tool.

Lesson: *Monitoring isn't "install Prometheus". It's a dashboard that a specific person opens every morning. If it can't answer "is everything OK?" in 5 seconds — it's useless.*

03. Diagnostic agent on a local model

Challenge: Data from the bank's closed network can't go to the cloud. Simple question: can a local model handle the on-call role — go into the logs, metrics, and runbooks on its own, gather the facts, and reach a diagnosis.

Solution: Built a PoC: an agent with a toolset, a registry of gathered facts, and answer validation. Ran Qwen3 (8B / 14B / 30B) against cloud DeepSeek as the quality ceiling. Added guard rules against the worst failure mode — when the model hasn't looked at anything but cheerfully reports "all clear".

Result: 14B without reasoning scored 6 out of 6 — same as the cloud. 8 vCPUs and 24 GB, no GPU, ~12 minutes per scenario. The architect signed off on the configuration; we're moving to a pilot on live data.

Lesson: *Model size doesn't fix honesty: the 30B confidently wrote "healthy" without calling a single tool. What saved us were boring guard rules, not parameters.*

---

04. Infrastructure you can just ask

Challenge: The team keeps coming to the DevOps engineer with the same questions: how does this pipeline work, how is the version computed, where do I look. The answers exist — but they're buried in Confluence, in Jenkinsfiles, and in my head. The classic "ask Maks".

Solution: Pulled the JD documentation into one place and put an MCP server with vector search on top of it. Both people and LLM agents query it: a specific question gets a specific answer with steps — no digging through ten pages.

Result: Live beta. Coverage is still growing, but some questions already go to the docs instead of to me. Agents pull context from there too — and make things up noticeably less.

Lesson: *It turned out not to be about the model. Boring docs aimed at a specific question beat a smarter model with no context — every single time.*

---

05. Jenkins as the CI/CD backbone for an enterprise project

Challenge: A complex product with many components, multiple OSes, lots of environments. Needed reliable and fast CI/CD.

Solution: Complex pipelines for the full cycle. Parallel builds, dynamic workers in vSphere. Active Choice UI. Auto-tests in PRs. OWASP Dependency Check.

Result: The team deploys on their own. PRs don't merge without tests. Dependencies are automatically checked for vulnerabilities.

Lesson: *Jenkins is powerful but dangerous. Without discipline, pipelines turn into unreadable Groovy scripts.*

---

06. Production MongoDB upgrade (EGRN)

Challenge: Upgrade MongoDB in production on one of the largest government IT projects. Downtime is critical, data loss is unacceptable.

Solution: Ansible playbooks for the upgrade. A production-like environment for full dry runs. Debugging until fully reproducible.

Result: Successful migration with zero data loss and minimal downtime.

Lesson: *In production there's no "let's try". Every step is verified on a staging environment. Playbooks are documentation that also executes.*

SKILLS

Core: Jenkins, GitLab CI/CD, Ansible, Docker, Prometheus, Grafana, HashiCorp Vault, VMware vSphere, Linux, PostgreSQL

Extended: Thanos, Packer, OpenSearch, Loki, Promtail, Logstash, Telegraf, Kafka, Keycloak, Nginx, Kubernetes, Terraform, Go, Python, Bash, Groovy, MinIO/S3, Apache Superset, Git

AI / LLM: MCP servers, Agent harnesses, LLM evals (promptfoo), LLM observability (Phoenix), Local inference, Claude Code, OpenAI API

Previously: Flask, MongoDB, HTML/JS/CSS, DigitalOcean, Ruby, Java, SonarQube

LANGUAGES

Russian: native

English: advanced (reading, listening; working on speaking)

EDUCATION

Yandex Practicum — DevOps for Operations and Development (including Kubernetes) (2022–2023)

Coursera — An Introduction to Interactive Programming in Python (2014)

Pedagogical College No. 1 (K.D. Ushinsky) — Social Pedagogy (2006–2009)